

SQL Aggregation Basics: 10 Queries

Functions covered: COUNT, SUM, AVG, MIN, MAX | Dataset: Northwind-style (7 tables) | 30 September 2026

Objective

Learn aggregate functions on the same cleaned dataset used in the earlier projects. Half the queries aggregate the whole table, and half use GROUP BY to aggregate per category, per order, per customer or per supplier, so the difference between a single summary number and a grouped summary is clear.

Queries and outputs

Query 1: Total number of products (COUNT)

```
SELECT COUNT(*) AS total_products
FROM Products;
```

total_products
25

1 row(s) returned.

Query 2: Number of products per category (COUNT + GROUP BY)

```
SELECT CategoryID, COUNT(*) AS product_count
FROM Products
GROUP BY CategoryID
ORDER BY CategoryID;
```

CategoryID	product_count
1	7
2	11
3	1
4	2
5	4

5 row(s) returned.

Query 3: Total quantity ordered across all order lines (SUM)

```
SELECT SUM(Quantity) AS total_quantity
FROM OrderDetails;
```

total_quantity
306

1 row(s) returned.

Query 4: Total quantity ordered per order (SUM + GROUP BY)

```
SELECT OrderID, SUM(Quantity) AS total_quantity
FROM OrderDetails
GROUP BY OrderID
ORDER BY total_quantity DESC;
```

OrderID	total_quantity
10249	49
10254	36

OrderID	total_quantity
10250	28
10251	27
10248	27
10256	18
10261	16
10253	16
10265	14
10264	12
10260	11
10252	10
10266	9
10263	8
10262	7
10259	6
10255	5
10257	4
10258	3

19 row(s) returned.

Query 5: Average price of all products (AVG)

```
SELECT AVG(Price) AS avg_price
FROM Products;
```

avg_price
15.69

1 row(s) returned.

Query 6: Average product price per category (AVG + GROUP BY)

```
SELECT CategoryID, AVG(Price) AS avg_price
FROM Products
GROUP BY CategoryID
ORDER BY avg_price DESC;
```

CategoryID	avg_price
2	16.77
1	16.36
4	16.35
3	16.00
5	11.12

5 row(s) returned.

Query 7: Cheapest and priciest product overall (MIN, MAX)

```
SELECT MIN(Price) AS min_price, MAX(Price) AS max_price
FROM Products;
```

min_price	max_price
2.70	30.00

1 row(s) returned.

Query 8: Price range per category (MIN, MAX + GROUP BY)

```
SELECT CategoryID, MIN(Price) AS min_price, MAX(Price) AS max_price
FROM Products
GROUP BY CategoryID;
```

CategoryID	min_price	max_price
1	9.50	22.00
2	7.20	30.00
3	16.00	16.00
4	2.70	30.00
5	3.50	25.00

5 row(s) returned.

Query 9: Number of orders per customer (COUNT + GROUP BY)

```
SELECT CustomerID, COUNT(*) AS order_count
FROM Orders
GROUP BY CustomerID
ORDER BY order_count DESC;
```

CustomerID	order_count
99	1
90	1
89	1
88	1
81	1
80	1
78	1
76	1
73	1
66	1
54	1
50	1
45	1
41	1
39	1
34	1
33	1
25	1
22	1

CustomerID	order_count
12	1
11	1

21 row(s) returned.

Query 10: Suppliers with more than 2 products (COUNT, AVG + HAVING)

```
SELECT SupplierID, COUNT(*) AS product_count, AVG(Price) AS avg_price
FROM Products
GROUP BY SupplierID
HAVING COUNT(*) > 2
ORDER BY product_count DESC;
```

SupplierID	product_count	avg_price
5	7	19.92
2	6	18.31
1	6	13.75
4	3	13.16
3	3	6.97

5 row(s) returned.

What I noticed testing each function

- **COUNT(*)** counts every row in the group, including rows with NULLs in other columns; **COUNT(column)** would skip NULLs in that specific column.
- **SUM** and **AVG** only work on numeric columns, and both ignore NULLs automatically — a NULL is left out of the total and the count used to divide, not treated as zero.
- **GROUP BY** has to include every non-aggregated column in the SELECT list, or the query is invalid in PostgreSQL (MySQL is more lenient but it's still bad practice).
- **HAVING** filters groups after aggregation, while **WHERE** filters rows before aggregation — that's why query 10 uses **HAVING** to keep only suppliers with more than 2 products.

Interview questions

Why use GROUP BY? It splits rows into groups that share the same value in one or more columns, so an aggregate function (COUNT, SUM, AVG, MIN, MAX) is calculated separately for each group instead of across the whole table. For example, grouping Products by CategoryID gives an average price per category rather than one overall average.

How does AVG handle nulls? It ignores NULL values completely — they are excluded from both the sum and the count used to calculate the average. So AVG over a column with some NULLs is the average of only the non-NULL rows, not an average that treats NULLs as zero.